Trabajo Práctico Integrador Programación I
Sistema de Control de Inventario

Alumna
Avila Ruiz Eugenia

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

08 de Noviembre de 2026

Introducción **3**
Conceptos aplicados **3**
1. Listas 3
2. Diccionarios 4
3. Funciones 4
4. Condicionales 5
5. Bucles 6
6. Manejo de errores con try/except/raise 7
Diagrama de flujo **9**
Lenguaje, entorno y recursos **10**
Python 10
Visual Studio Code 10
Git y GitHub 10
Mermaid 10
Conclusión **11**
Bibliografía **12**
Anexo **13**

Introducción

En este informe se describe el desarrollo de un programa realizado a partir de los requerimientos del trabajo integrador. El programa es un sistema de control de inventario para una ferretería, desarrollado en Python. Permite cargar herramientas con su stock inicial, ver el inventario completo, consultar el stock de una herramienta, conocer cuáles están agotadas, dar de alta productos nuevos y registrar ventas e ingresos de mercadería.

Los datos se guardan en una lista de diccionarios, donde cada diccionario representa una herramienta con su nombre y su cantidad. Cada opción del menú está resuelta en su propia función, y el programa maneja los errores sin interrumpirse: si el usuario ingresa un dato inválido, se le informa el motivo y puede seguir usándolo.

En las secciones siguientes se desarrollan los conceptos que sirvieron de base para el programa (listas, diccionarios, funciones, condicionales, bucles y manejo de errores), el diagrama de flujo de las operaciones principales y el lenguaje y el entorno de desarrollo utilizados.

Conceptos aplicados

1. Listas

Según el material de la cátedra, una lista es “una estructura de datos que permite almacenar una secuencia ordenada de elementos”, que además es mutable, es decir, puede modificarse después de ser creada (Programación I, UTN).

En el programa se usa una única lista, llamada inventario, donde cada elemento es un diccionario que representa una herramienta. La lista se crea vacía dentro de la función main() (ver Figura 1 en el Anexo) y no fuera de ella, porque el programa no utiliza variables globales: desde main() se pasa como parámetro a cada función del menú.

Las herramientas se agregan al final de la lista con el método append() en las opciones 1 (carga de herramientas) y 5 (alta de nuevo producto). Para mostrarlas, buscarlas o encontrar las que están agotadas, la lista se recorre con un for, en la función auxiliar buscar_herramienta y en las opciones 2 y 4. Además, con len(inventario) == 0 se comprueba si la lista está vacía antes de pedir datos al usuario en las opciones 2, 3, 4 y 6.

Como las listas son mutables, cuando una función agrega una herramienta o modifica su stock, el cambio queda guardado en la misma lista que creó main(). Así, lo que se carga en una opción está disponible para todas las demás.

2. Diccionarios

Según el material de la cátedra, un diccionario “almacena pares clave–valor. En lugar de acceder por posición (como en las listas), accedemos por una clave única” (Programación I, UTN).

En el programa, cada herramienta es un diccionario con dos claves: 'herramienta', que guarda el nombre de la herramienta, y 'cantidad', que guarda su stock. Todos estos diccionarios se guardan dentro de la lista inventario.

Los diccionarios se crean en las opciones 1 y 5, armándolos directamente dentro de append() (ver Figura 2 en el Anexo).

Se hace así porque el enunciado no permite crear diccionarios sueltos fuera de la lista inventario, así que cada diccionario nuevo se arma en el mismo append(). De esta manera, todos los datos de las herramientas quedan guardados en un solo lugar.

Para leer un dato se usa la clave entre corchetes, por ejemplo producto['cantidad']. Esto se hace en las opciones 2, 3, 4 y 6, y en la función auxiliar buscar_herramienta, que compara producto['herramienta'] con el nombre buscado.

En la opción 6, el valor de 'cantidad' se resta cuando hay una venta y se suma cuando hay un ingreso. Como producto apunta al mismo diccionario que está dentro de la lista, el cambio de stock queda guardado directamente en la lista inventario, sin necesidad de volver a agregar el diccionario ni de devolverlo con return. Por ejemplo, si se venden todas las unidades de una herramienta, la opción 4 ya la muestra como agotada.

Se eligió un diccionario porque, como dice la cátedra, permite asociar un dato con otro: en este caso, el nombre de cada herramienta con su stock. Así, el stock se obtiene por el nombre de la clave ('cantidad') y no por una posición, como ocurre en las listas.

3. Funciones

Según el material de la cátedra, una función es “un bloque de código con nombre que realiza una tarea específica. La idea es escribir ese bloque una sola vez y después

llamarlo (usarlo) cada vez que lo necesitemos, en lugar de repetir el mismo código una y otra vez" (Programación I, UTN).

El programa está organizado en nueve funciones. Cada opción del menú, de la 1 a la 6, tiene su propia función; por ejemplo, cargar_herramientas resuelve la opción 1 y actualizar_stock la opción 6. La función main() crea la lista inventario, muestra el menú hasta que el usuario elige la opción 7 y, según la opción elegida, llama a la función correspondiente (ver Figura 3 en el Anexo). También captura los errores que llegan desde las opciones, muestra el motivo y vuelve al menú.

Además, se crearon dos funciones auxiliares que se reutilizan en las opciones 1, 3, 5 y 6. La función validar_nombre le quita al nombre los espacios del principio y del final, lanza un error si queda vacío o si no contiene ninguna letra y, si es válido, lo devuelve limpio con return (ver Figura 4 en el Anexo). La función buscar_herramienta busca una herramienta por su nombre sin tener en cuenta mayúsculas ni espacios sobrantes, y devuelve el diccionario de la herramienta si la encuentra o None si no existe.

Todas las funciones del menú reciben la lista inventario como parámetro, ya que el programa no usa variables globales. En varias de ellas también se usa return sin ningún valor, para terminar la función antes de tiempo cuando el inventario está vacío y volver al menú sin pedir datos al usuario.

Dividir el programa en funciones aplica directamente la idea de la cátedra: el control del nombre (que no esté vacío y que tenga al menos una letra) está escrito una sola vez en validar_nombre, en lugar de repetirse en cuatro opciones. Además, cada función tiene una tarea clara, lo que hace que el código sea más fácil de leer, de probar y de corregir.

4. Condicionales

Según el material de la cátedra, las estructuras condicionales son "mecanismos de control que permiten a los algoritmos tomar decisiones basadas en si se cumple o no una condición. Se podría pensar en ellas como 'caminos alternativos' para el fluir de los datos dentro de un programa" (Programación I, UTN).

En el programa, los condicionales abren esos caminos alternativos según lo que ingresa el usuario. En main(), una cadena de if/elif compara la opción elegida y llama a la función que corresponde (ver Figura 3 en el Anexo). El último caso, elif opcion != 7, detecta cualquier número que no esté entre 1 y 7 y lanza un error de opción fuera de rango.

Los condicionales también se usan para validar los datos antes de usarlos. Por ejemplo, en la opción 1 se controla que la cantidad de herramientas a cargar sea mayor que 0 (cantidad_herramientas <= 0); en las opciones 1 y 5, que el stock inicial no sea negativo (stock < 0); y en la opción 6, que las unidades del movimiento sean mayores que 0 (unidades <= 0) y que, en una venta, haya stock suficiente (unidades > producto['cantidad']). Si alguna de estas condiciones se cumple, se lanza un error y el dato no se acepta.

En la opción 6 también se usa el operador and para comprobar que el tipo de movimiento sea válido: la condición opcion != "1" and opcion != "2" se cumple cuando lo ingresado no es ninguna de las dos opciones, y en ese caso se informa con un mensaje. Después, un if/else distingue entre venta y ingreso: si es una venta, el stock se resta; si no, se suma.

En la opción 4, el condicional if not hay_agotados se evalúa después de recorrer la lista y permite mostrar un mensaje propio cuando ninguna herramienta tiene stock cero.

Por último, los casos informativos, como un inventario vacío o una herramienta que no existe, se resuelven solo con if y print, sin lanzar errores, porque no son datos inválidos sino situaciones que se le informan al usuario.

5. Bucles

Según el material de la cátedra, los bucles “le dicen a la computadora que repita un bloque de instrucciones muchas veces, sin que tengamos que escribirlo una y otra vez. […] Hay dos grandes tipos: los que se repiten mientras se cumpla una condición (while), y los que recorren cada elemento de una colección (for)” (Programación I, UTN). En el programa se usan los dos tipos.

El bucle principal está en main(): while opcion != 7 vuelve a mostrar el menú después de cada opción, mientras el usuario no elija salir (ver Figura 3 en el Anexo). Esto es lo que hace que el menú sea persistente. Cuando se elige la opción 7, la condición deja de cumplirse, el bucle termina y el programa finaliza.

El segundo tipo, el for que recorre una colección, se usa con la lista inventario. La instrucción for producto in inventario recorre una por una las herramientas guardadas: en buscar_herramienta para encontrar una por su nombre, en mostrar_inventario para listarlas y en reporte_agotados para detectar las que tienen stock cero.

La función cargar_herramientas combina los dos tipos (ver Figura 2 en el Anexo). Primero, un bucle while True pide la cantidad de herramientas a cargar. Si el usuario ingresa un número menor o igual que 0, se lanza un ValueError con raise; si ingresa un valor que no es un número entero, como abc o 3.5, el error lo lanza int(). En ambos casos el except muestra el motivo y el bucle vuelve a pedir la cantidad. Solo cuando el dato es válido se ejecuta el break, que corta el bucle.

Después, for i in range(cantidad_herramientas) repite la carga tantas veces como indicó el usuario. En cada iteración se usan dos bucles while True separados, con el mismo mecanismo de reintento. El primero pide el nombre y lo vuelve a pedir si está vacío, si no contiene ninguna letra o si la herramienta ya existe en el inventario. El segundo pide el stock inicial y lo vuelve a pedir si no es un número entero o si es negativo; el 0 es válido. Se usan dos bucles separados para que, si el stock es inválido, se vuelva a pedir solo el stock, sin perder el nombre ya ingresado. Recién cuando los dos datos son válidos, la herramienta se agrega a la lista.

6. Manejo de errores con try/except/raise

Según el material de la cátedra, “la herramienta central es la estructura try / except. El código «riesgoso» (el que podría fallar) se pone dentro del try. Si ocurre una excepción, en lugar de romperse, Python salta al bloque except, que decide qué hacer”. Además, al capturar una excepción se la puede guardar en una variable con as para ver su mensaje, y se pueden lanzar errores a propósito con raise (Programación I, UTN).

En el programa, todos los errores son de tipo ValueError, y aparecen de dos formas. La primera es automática: cuando el usuario escribe algo que no es un número entero, como abc o 3.5, la función int() no puede convertirlo y lanza el error. La segunda es a propósito, con raise, cuando un dato rompe una regla del sistema: un nombre vacío o sin ninguna letra (ver Figura 4 en el Anexo), una herramienta repetida, un stock negativo, una cantidad o unidades menores o iguales que 0, una opción de menú fuera de rango o una venta mayor que el stock disponible. La cátedra menciona que también se pueden crear excepciones propias con una clase, pero el enunciado no permite usar clases, por eso se usa ValueError, que ya viene incluido en Python.

Cada error se captura en un lugar distinto según lo que tiene que pasar después. En main(), el try rodea la lectura de la opción y la llamada a cada función. Si una opción lanza un error, como en las opciones 3, 5 y 6, la función se interrumpe, el except de main() lo captura, muestra el motivo y el programa vuelve al menú. En cambio, en la opción 1 el try/except está dentro de la función, en cada bucle de carga (ver Figura 2 en

el Anexo): el error se captura ahí mismo y se vuelve a pedir solo el dato inválido, sin volver al menú.

En los dos casos se usa except ValueError as e y luego print(f"Error: {e}"), de modo que el usuario ve el mensaje exacto del error, por ejemplo “Error: El stock no puede ser negativo.”. Como todas las validaciones se hacen antes de modificar el inventario, cuando hay un error no se agrega ninguna herramienta ni cambia ningún stock.

Por último, hay situaciones que no son errores sino casos informativos: un inventario vacío, una herramienta que no existe o un tipo de movimiento inválido. Estos se resuelven con un if y un mensaje, sin raise. Gracias a este manejo de errores, ningún dato inválido interrumpe el programa.

Diagrama de flujo

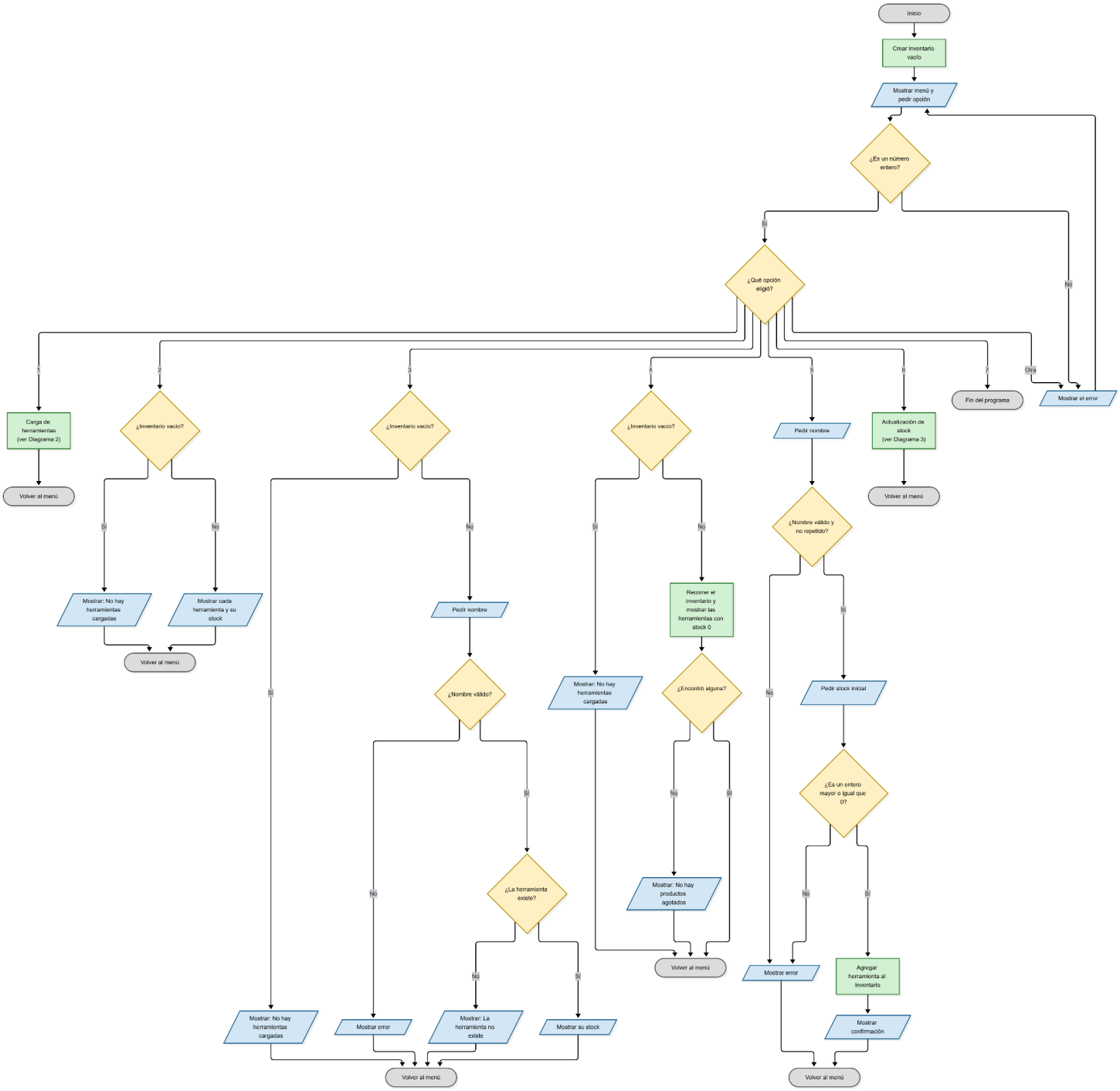


Diagrama 1. Flujo general del programa: menú principal y opciones. Versión en tamaño completo:

trabajo_practico_integrador_programacion/diagramas/diagrama_general.png at abc81a48645812717c31519d3c62922f094e22d1 · Eugeniaavilaruiz/trabajo_practico_integrador_programacion

Lenguaje, entorno y recursos

Python.

Es un lenguaje de programación de alto nivel, interpretado y de propósito general, con una sintaxis simple que se basa en la indentación del código. El programa se desarrolló con Python 3.14. Se eligió porque es el lenguaje que se utiliza en la materia y porque sus listas y diccionarios permiten representar el inventario de forma directa. El programa está formado por un solo archivo, codigo_fuente_trabajo_integrador_programacion.py, que es el que se ejecuta para iniciarlo.

Visual Studio Code.

Es un editor de código gratuito desarrollado por Microsoft, que se adapta a distintos lenguajes mediante extensiones. Se usó la versión 1.145.2 , junto con la extensión de Python de Microsoft, que agrega el resaltado de sintaxis, el autocompletado y la ejecución del programa desde el mismo editor. Se eligió porque es liviano, gratuito y tiene una terminal integrada, lo que permitió escribir, ejecutar y probar el programa en un solo lugar.

Git y GitHub.

Git es un sistema de control de versiones que registra los cambios de un proyecto y permite volver a versiones anteriores. Se usó la versión 2.54.0. GitHub es una plataforma web que almacena repositorios de Git en la nube. Se eligieron para guardar una copia de respaldo del código y tener un historial de cada avance: el esqueleto del menú, las funciones auxiliares y cada una de las opciones.

Mermaid.

Es un lenguaje de código abierto que permite crear diagramas a partir de una descripción en texto: cada figura y cada flecha se escriben como una línea, y la

herramienta genera el dibujo automáticamente. Los diagramas de flujo del informe se hicieron con Mermaid Live Editor (mermaid.live), su editor en línea, desde donde se exportaron como imágenes. Se eligió porque es gratuito, no requiere instalación y, al estar el diagrama escrito como texto, resulta sencillo corregirlo si cambia el programa. Además, GitHub reconoce este formato, por lo que los diagramas también se guardaron en el repositorio del proyecto.

Conclusión

El trabajo permitió desarrollar un sistema de control de inventario que cumple con todas las operaciones pedidas: cargar herramientas, consultar el stock, detectar las agotadas, dar de alta productos y registrar ventas e ingresos, sin que ningún dato inválido interrumpa el programa.

Más allá del resultado, el desarrollo sirvió para entender cómo se combinan los conceptos de la materia en un programa real. La lista de diccionarios resultó una forma simple de organizar los datos, y dividir el programa en funciones, con dos funciones auxiliares reutilizables, evitó repetir código y facilitó encontrar errores. El manejo de errores fue la parte que más análisis requirió: no alcanza con lanzar un error, también hay que decidir dónde capturarlo, según si conviene volver al menú o volver a pedir el mismo dato.

También quedó clara la importancia de probar el programa con casos límite. Varias fallas aparecieron recién al probar valores como 0, números negativos o nombres formados solo por símbolos: por ejemplo, una comparación con el signo invertido que rechazaba ventas válidas, o la diferencia entre = y ==, que hacía que una bandera nunca cambiara de valor. Corregir esos errores ayudó a entender con más precisión qué hace cada línea del código.

Por último, el uso de herramientas como Git y GitHub para guardar cada avance, y de Mermaid para representar el flujo del programa, mostró que programar no es solo escribir código, sino también organizarlo, documentarlo y poder explicarlo.

Bibliografía

García, M. A. (2026). Programación 1 — Unidad 3: Estructuras repetitivas. Actividad 1: Introducción a estructuras repetitivas (while, for) [Manual de lectura]. Universidad Tecnológica Nacional.

García, M. A. (2026). Programación 1 — Unidad 6: Funciones. Actividad 1: Introducción a funciones y diferencias con procedimientos [Manual de lectura]. Universidad Tecnológica Nacional.

García, M. A. (2026). Programación 1 — Unidad 8: Manejo de errores. Actividad 2: Excepciones en Python [Manual de lectura]. Universidad Tecnológica Nacional.

Universidad Tecnológica Nacional. (s. f.). Estructuras de datos en Python [Material de la cátedra]. Tecnicatura Universitaria en Programación.

Universidad Tecnológica Nacional. (s. f.). Introducción a Git y GitHub: tu máquina del tiempo de código [Material de la cátedra].

Universidad Tecnológica Nacional. (2026). Trabajo Práctico Integrador: Sistema de Control de Inventario [Enunciado]. Cátedra Programación 1.

Python Software Foundation. (2025). Documentación de Python 3.14. https://docs.python.org/es/3/

Anexo

```python
def main():
    """Función principal: crea el inventario y muestra el menú hasta elegir la opción 7.

    Captura los ValueError que llegan de las opciones (menú inválido y opciones 3, 5 y 6),
    muestra el motivo y vuelve a mostrar el menú.
    """
    inventario = []
    opcion = 0
    print('Sistema de inventario iniciado')
    while opcion != 7:
        try:
            print("-----MENÚ-----")
            print("1. Carga de herramientas")
            print("2. Visualización de inventario")
            print("3. Consulta de stock")
            print("4. Reporte de agotados")
            print("5. Alta de nuevo producto")
            print("6. Actualización de stock (venta / ingreso)")
            print("7. Salir")
            opcion = int(input("Seleccione una opción: "))
            if opcion == 1:
                cargar_herramientas(inventario)
            elif opcion == 2:
                mostrar_inventario(inventario)
            elif opcion == 3:
                consultar_stock(inventario)
            elif opcion == 4:
                reporte_agotados(inventario)
            elif opcion == 5:
                alta_producto(inventario)
            elif opcion == 6:
                actualizar_stock(inventario)
            elif opcion != 7:
                raise ValueError("Opción fuera de rango (1 a 7).")
        except ValueError as e:
            print(f"Error: {e}")
    print("Fin del programa")
```

Figura 1. Creación de la lista inventario dentro de main().

```python
def cargar_herramientas(inventario):
    """Opción 1: carga varias herramientas con su stock inicial.

    Pide cuántas herramientas se van a cargar y, por cada una, el nombre y el stock.
    Los errores se capturan dentro de la función y se vuelve a pedir solo el dato inválido.
    Cada herramienta se agrega a inventario recién cuando el nombre y el stock son válidos.
    Parámetro: inventario (lista de herramientas).
    """
    while True:
        try:
            cantidad_herramientas = int(input("¿Cuántas herramientas vas a cargar? "))
            if cantidad_herramientas <= 0:
                raise ValueError("La cantidad debe ser un número mayor que 0.")
            break
        except ValueError as e:
            print(f"Error: {e}")
    for i in range(cantidad_herramientas):
        print(f"--- Herramienta {i + 1} de {cantidad_herramientas} ---")
        while True:
            try:
                nombre = validar_nombre(input("Nombre: "))
                if buscar_herramienta(nombre, inventario) != None:
                    raise ValueError(f"La herramienta '{nombre}' ya existe en el inventario.")
                break
            except ValueError as e:
                print(f"Error: {e}")
        while True:
            try:
                stock = int(input(f"Stock inicial de {nombre}: "))
                if stock < 0:
                    raise ValueError("El stock no puede ser negativo.")
                break
            except ValueError as e:
                print(f"Error: {e}")
        inventario.append({'herramienta': nombre, 'cantidad': stock})
        print(f"Se dio de alta '{nombre}' con {stock} unidades.")
```

Figura 2. Creación del diccionario dentro de append() en la opción 1.

```python
def main():
    """Función principal: crea el inventario y muestra el menú hasta elegir la opción 7.

    Captura los ValueError que llegan de las opciones (menú inválido y opciones 3, 5 y 6),
    muestra el motivo y vuelve a mostrar el menú.
    """
    inventario = []
    opcion = 0
    print('Sistema de inventario iniciado')
    while opcion != 7:
        try:
            print("-----MENÚ-----")
            print("1. Carga de herramientas")
            print("2. Visualización de inventario")
            print("3. Consulta de stock")
            print("4. Reporte de agotados")
            print("5. Alta de nuevo producto")
            print("6. Actualización de stock (venta / ingreso)")
            print("7. Salir")
            opcion = int(input("Seleccione una opción: "))
            if opcion == 1:
                cargar_herramientas(inventario)
            elif opcion == 2:
                mostrar_inventario(inventario)
            elif opcion == 3:
                consultar_stock(inventario)
            elif opcion == 4:
                reporte_agotados(inventario)
            elif opcion == 5:
                alta_producto(inventario)
            elif opcion == 6:
                actualizar_stock(inventario)
            elif opcion != 7:
                raise ValueError("Opción fuera de rango (1 a 7).")
        except ValueError as e:
            print(f"Error: {e}")
    print("Fin del programa")


main()
```

Figura 3. Llamadas a las funciones de cada opción desde main().

```python
def validar_nombre(nombre):
    """Valida el nombre de una herramienta ingresado por el usuario.

    Le quita los espacios del principio y del final.
    Parámetro: nombre (texto escrito por el usuario).
    Devuelve: el nombre sin espacios sobrantes.
    Lanza ValueError si el nombre queda vacío o si no contiene ninguna letra.
    """
    nombre = nombre.strip()
    if nombre == "":
        raise ValueError("El nombre de la herramienta no puede estar vacío.")
    tiene_letra = False  # bandera
    for caracter in nombre:
        if caracter.isalpha():
            tiene_letra = True
    if not tiene_letra:
        raise ValueError("El nombre de la herramienta debe contener al menos una letra.")
    return nombre
```

Figura 4. Función auxiliar validar_nombre(), reutilizada en las opciones 1, 3, 5 y 6.